LearnMap.app — Design Document (MVP v1)

1. Overview

Project Name

LearnMap.app

Vision

LearnMap is a personal learning operating system that allows users to organize what they want to learn, track learning progress, manually log study sessions, and visualize learning statistics.

The MVP intentionally excludes AI. Instead, it focuses on collecting structured data that can later power intelligent coaching and progress analysis.

The application is designed for a single user and stores all data locally using SQLite.

2. Goals

Primary Goals

Organize learning into a hierarchy

Track completion

Track manual study hours

Visualize progress

Be enjoyable enough to use daily

Non Goals

No authentication

No cloud sync

No collaboration

No notifications

No AI

No quizzes

No note editor

No mobile app

3. Design Philosophy

The application should feel like:

"Duolingo meets Linear meets Notion"

Characteristics:

clean

spacious

playful

fast

minimal clicks

subtle animations

dopamine without becoming distracting

Everything should feel lightweight.

4. Theme

Primary Theme

Light

No dark mode in MVP.

Accent Color

Warm Yellow

Suggested palette:

Primary

#FACC15

Hover

#EAB308

Soft Background

#FEF9C3

Cards

#FFFFFF

Background

#FFFDF7

Neutral Colors

Background

#FFFDF7

Secondary Background

#FAFAF5

Borders

#EAE7DC

Text

#222222

Secondary Text

#666666

Success

Green

#22C55E

Warning

Orange

#F97316

Error

Red

#EF4444

5. Typography

Headings

Poppins

Body

Inter

Numbers

JetBrains Mono

6. Design Language

Rounded corners

16px

Buttons

Rounded

Soft shadows

Floating cards

Lots of whitespace

Smooth hover animations

Micro interactions

Examples:

✓ checkbox pops

Cards lift slightly

Buttons scale

Charts animate

Progress bars fill smoothly

7. Tech Stack

Frontend

React

TypeScript

Vite

TailwindCSS

React Router

TanStack Query

Axios

shadcn/ui

Recharts

React Hook Form

Zod

Backend

Go

Gin

GORM

SQLite

Air (hot reload)

8. Folder Structure

learnmap/

  frontend/

  backend/

  docs/

  README.md

  docker-compose.yml

Frontend

src/

  components/

pages/

hooks/

services/

layouts/

routes/

types/

utils/

Backend

cmd/

internal/

handlers/

services/

repositories/

models/

routes/

middleware/

database/

9. Core Features

Feature 1

Learning Tree

Users can create:

Backend

Java

Collections

Streams

JVM

Spring

MVC

Security

Unlimited nesting.

Feature 2

Task Management

Each learning item has

Title

Description

Status

Deadline

Created Date

Completed Date

Feature 3

Study Sessions

Manual logging.

Example

Topic

Kafka

Hours

2.5

Date

Today

Notes

Worked through Consumer Groups.

Feature 4

Dashboard

Cards

Study Hours This Week

Current Streak

Topics

Completed Items

Pending Items

Charts

Weekly Hours

Monthly Hours

Most Studied Topics

Completion %

10. Dashboard Layout

Header

Study Hours

Current Streak

Completed

Pending

Weekly Hours Chart

Top Topics

Today's Sessions

Recent Activity

---

11. Pages

Dashboard

Landing page.

Contains:

Statistics

Charts

Activity

Quick Add

Learning

Tree View

Expand/collapse hierarchy

Add Child

Delete

Rename

Mark Complete

Study Sessions

Table

Date

Topic

Hours

Notes

Button

+ Add Session

Statistics

Graphs

Weekly

Monthly

Yearly

12. Database

learning_items

id

parent_id

title

description

status

deadline

created_at

updated_at

completed_at

study_sessions

id

learning_item_id

hours

notes

session_date

created_at

events

Future AI support.

id

event_type

entity_type

entity_id

payload

created_at

MVP won't use this table directly.

13. REST APIs

Learning Items

GET /items

POST /items

PUT /items/:id

DELETE /items/:id

Sessions

GET /sessions

POST /sessions

DELETE /sessions/:id

Dashboard

GET /dashboard

Returns

Study hours

Weekly chart

Streak

Completed

Pending

14. UX Details

Every page has

Search bar

Breadcrumb

Floating Add button

Deleting requires confirmation.

Completed items become

Light Green

with

✓

Collapsed nodes remember state.

Animations

150–200ms.

Never flashy.

15. Future AI Compatibility

Even though AI is excluded,

the architecture should already capture events.

Example:

TASK_CREATED

TASK_COMPLETED

TASK_REOPENED

SESSION_ADDED

ITEM_RENAMED

Later,

Claude/OpenAI can reconstruct the entire learning history by replaying these events.

16. MVP Roadmap

Milestone 1

Backend

SQLite setup

GORM models

CRUD APIs

Milestone 2

Frontend

Layout

Navigation

Dashboard

Learning Tree

Milestone 3

Study Sessions

Add

Delete

Statistics

Milestone 4

Charts

Weekly Hours

Completion %

Top Topics

Milestone 5

Polish

Animations

Responsive UI

Empty states

Loading states

Error handling

17. Success Criteria

The MVP is considered successful if:

You can create and organize a learning hierarchy.

You can mark items complete and see progress reflected immediately.

You can manually log study sessions with hours and notes.

The dashboard accurately summarizes weekly effort and completion metrics.

The application feels fast, visually inviting, and pleasant enough that you'd choose to open it every day rather than maintaining a spreadsheet or notes manually.

This establishes a solid foundation for future additions like AI coaching, quizzes, roadmap generation, and knowledge analytics without requiring architectural changes.